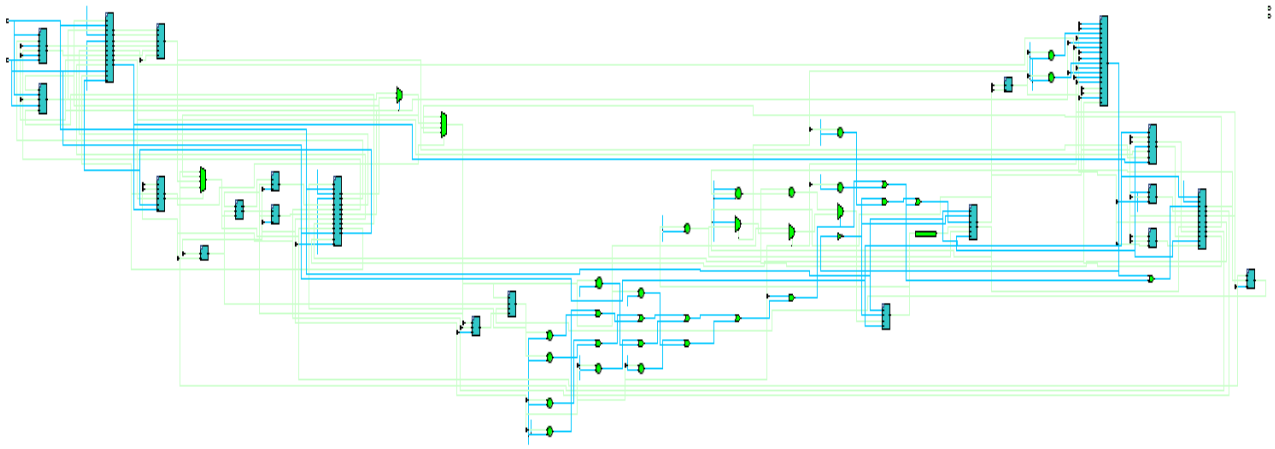
Design and Analysis of a MIPS Processor (w/ & w/o forwarding logic)



Introduction/Overview:

This report presents the design and implementation of element groupings of a Verilog-based simulation model of a 32-bit, 5-stage pipelined MIPS (Microprocessor w/o Interlocked Pipeline Stages) Processor. The design in question is fully outfitted to receive and execute instructions in the form of compiled MIPS Assembly (hexadecimal translation of assembly instructions). It also contains/initializes a 4KB word-addressed memory (RAM) and has a designated Hi/Lo unit (for mult/div + mtxx/mfxx instr.). To simulate pipelining, there are 4 “pipes” (pipeline modules), each of which dictates the flow of relevant instruction data from one cycle to the next. In the no-forwarding version of the processor, stalls are assessed any time an instruction in the decode stage requires data from a general-purpose register that is set to receive updates that has not yet exited the write-back stage. Stalls are also assessed whenever the Hi/Lo registers are accessed for the same stage/cycle splits. In the forwarding version of the processor, Hi/Lo stall behavior remains largely the same (diminishing returns to add proper forwarding), but general-purpose registers become eligible to receive updates as soon as the previous instruction reaches the memory stage (with memory given priority over write-back for forwarding). To handle forwarding behavior, a new “forwarding unit” must be instantiated, and the hazard unit has to be cut down accordingly as well (fewer stalls are necessary with forwarded data). Furthermore, several updates must be made to the CPU (master) module to properly assign forwarded values once the conditions for forwarding have been met and assessed by the forwarding unit. One such change includes the addition of an extra Write-Back-style Memory Forwarding MUX to ensure that data forwarded from the memory stage comes from the proper source (Data Memory, Hi/Lo Unit, ALU, PC+4). Beyond the general architecture, as evidenced in the non-single-file versions of the programs, the CPU module also contains outputs for debugging, which are useful for making waveform observations.

Main Control:

Arguably, the most important/interesting unit in the processor, this module is responsible for properly handling every instruction fed from the fetch stage and sending out processed bit-fields to every subsequent unit, so that they might operate as intended. For assigning instruction types (via opcodes and the funct field of the instruction), the following maps were referenced heavily:

ROOT

Table of opcodes for all instructions:

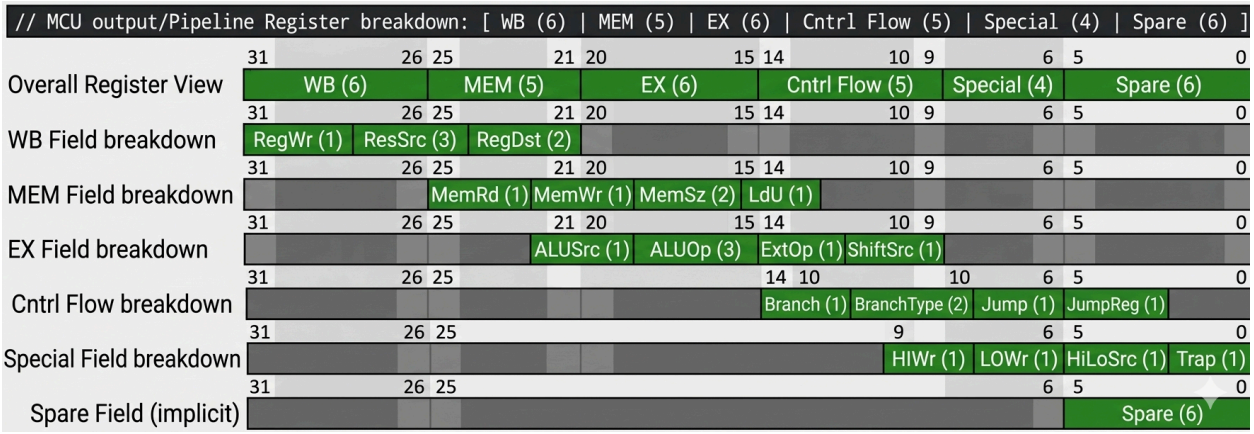
	000	001	010	011	100	101	110	111
000	REG		j	jal	beq	bne	blez	bgtz
001	addi	addiu	slti	sltiu	andi	ori	xori	
010								
011	llo	lhi	trap					
100	lb	lh		lw	lbu	lhu		
101	sb	sh		sw				
110								
111								

REG

Table of function codes for register-format instructions:

	000	001	010	011	100	101	110	111
000	sll		srl	sra	sllv		srlv	srav
001	jr	jalr						
010	mfhi	mthi	mflo	mtlo				
011	mult	multu	div	divu				
100	add	addu	sub	subu	and	or	xor	nor
101			slt	sltu				
110								
111								

The rationale behind the main-control processing was that it would handle/decode all the instructions that did not need to access the ALU or MDU (Mult/Div Unit), such as jumps, branches (partially, full handling of blez/bgtz), and move tos/froms. Furthermore, it was also intended to handle/simplify all I-type (immediate) instructions for quicker handling in the ALU. Throughout the process of assigning simplified opcode groupings and special case handling to incoming instructions, it became apparent that the MCU also needed to dictate the flow of data throughout the system, and thus, the following bit-field partitioning was conceived:

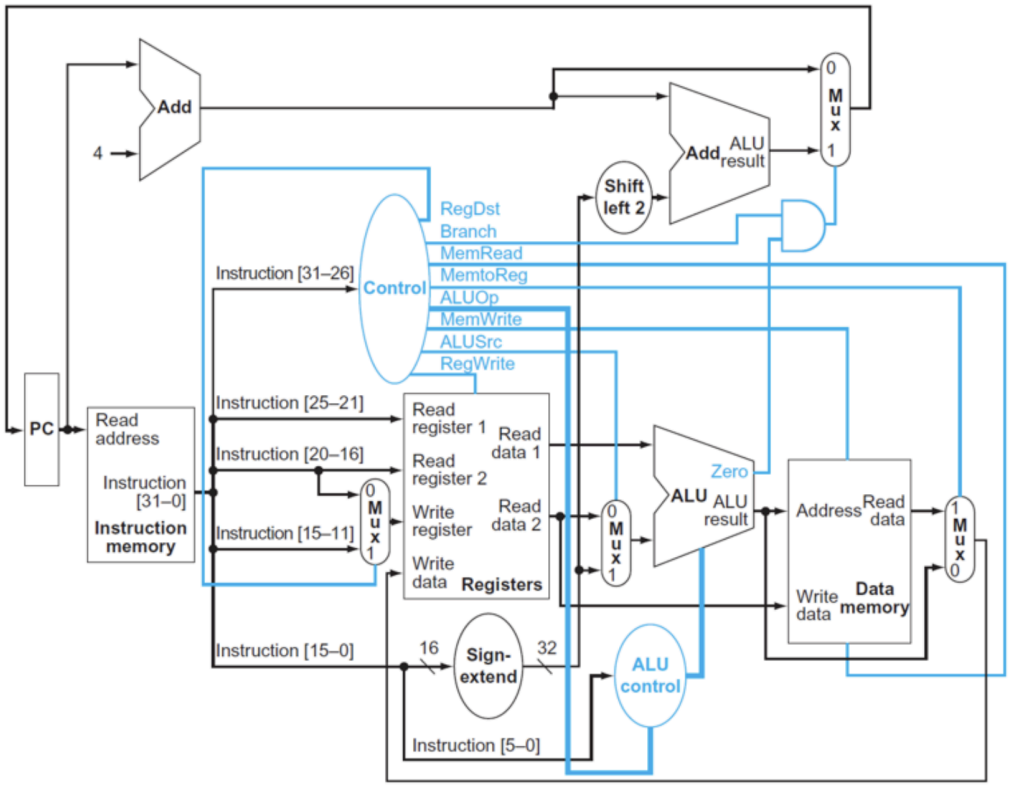


Due to the size and complexity of this main-control output register, you'll notice that the comments within the MCU module are by far the most detailed and educational you'll find in the entire file (I needed to reference it many, many times myself). Complexity aside, however, the mapping is intentionally "neat", as splitting it up into pipeline-stage sections helps give a general idea of the purpose of any arbitrary bit in the 32-bit array (initially 12-bits, but many more needs were realized during its construction). For a quick breakdown, reference the section below:

```
[31] RegWr
  1 = write to register file
  0 = no register write

[30:28] ResSrc (writeback source)
  000 = ALU result
  001 = Data memory
  011 = PC + 4
  100 = HI register
  101 = LO register

[27:26] RegDst (destination register)
  00 = rt (I-type)
  01 = rd (R-type)
  10 = $ra (jal, jalr)

[25] MemRd
  1 = memory read (load)
  0 = no read

[24] MemWr
  1 = memory write (store)
  0 = no write

[23:22] MemSz (memory access size)
  00 = byte
  01 = half-word
  10 = word

[21] LdU (load unsigned)
  1 = zero-extend
  0 = sign-extend

[20] ALUSrc
  1 = immediate operand
  0 = register operand

[19:17] ALUOp
  000 = AND   (and, andi)
  001 = OR    (or, ori)
  010 = ADD   (add, addi, addiu, load, store)
  011 = SUB   (sub, beq, bne)
  100 = XOR   (xor, xori)
  101 = SLT   (slt, slti)
  110 = SLTU  (sltu, sltiu)
  111 = R-type (use funct field)
```

```
[16] ExtOp
  1 = sign-extend immediate
  0 = zero-extend immediate

[15] ShiftSrc
  1 = shamt field
  0 = register value

[14] Branch
  1 = branch instruction
  0 = not a branch

[13:12] Branch Type
  00 = BEQ   (==)
  01 = BNE   (!=)
  10 = BLEZ  (<= 0)
  11 = BGTZ  (> 0)

[11] Jump
  1 = jump instruction
  0 = not a jump

[10] JumpReg
  1 = jump register (jr, jalr)
  0 = not jump register

[9] HIWr
  1 = write HI (mthi / mult/div)
  0 = no HI write

[8] LOWr
  1 = write LO (mtlo / mult/div)
  0 = no LO write

[7] HiLoSrc
  1 = write RS → HI/LO
  0 = write MDU → HI/LO

[6] Trap
  1 = trap instruction
  0 = not a trap
```

Except for slight adjustments to path logic (such as the extensions of MemtoReg and RegDst), or the lack of a Mult/Div Unit (and later Hi/Lo Unit), the typical MIPS processor dataflow diagram can be cross-referenced with the control word for better understanding:



Here I've included the single-cycle (non-pipelined version) because (in my opinion) it makes the functions of the MCU more obvious. The pipelined diagram can be similarly helpful, but it obfuscates the paths behind pipes (which we may reference later). Going over the mostly non-self-explanatory changes/modifications:

1. RegDst (Register Destination): controls whether or not the address of the destination register (register to be written to/updated) is **$t** or **$d**. Initially a single bit, this needed to be extended in the case of a link instruction to include **$ra/$31**. While **$31** is still a GPR, jump instructions typically do not include register fields, which makes this distinction necessary.
2. MemtoReg (Memory to Register?): controlled whether the data to be written back to the destination register was to be sourced from memory or ALU operations (load vs arithmetic). This was deemed incomplete due to the need to store **PC+4** for link instructions or **Hi/Lo** outputs for move-from instructions, which led to the creation of the 3-bit ResSrc (Result Source), which performs the same tasks with the expanded pathways. If an FPU were to be added, it would be very easy to add its write-back behavior to ResSrc.

3. MemSz (Memory Size): missing from MemWrite and MemRead, this controls whether to load/store bytes, halfwords, or words (simplifies opcode interpretation in Data Memory). Also missing from MemRead was a sign indicator, hence the addition of LdU (Load Unsigned?).
4. Branch Type: branch pathing formerly included only a "check if branch" control bit. Due to the various types of branching, this felt like a necessary inclusion, especially when considering that some branch operations don't require the ALU (technically, none do, but this design does not include branch predictions).

Other changes/modifications either expose inferred pathways (ExtOp - (Sign?) Extend Operand, ShiftSrc - Shift Source?, Jump - Jump?) or provide additional control features deemed appropriate for neat encoding. Presently, there is no support for LUI (load upper-immediate) style instructions, but if compatibility were to be added, it would likely just steal one of the spare bits. Also worth mentioning is the lack of support for multi-cycle instructions (PUSH, POP, MULT, etc.), but these are easily replicated using combinations of already supported operations.

Arithmetic Control:

Unlike the MCU, this module is only responsible for telling the ALU or MDU which arithmetic/logical operations to perform. Likewise, where the MCU receives full-size instructions from the fetch stage for execution, this module (which perhaps sits between decode and execute) only receives processed "ALU Opcodes" (ALUOp) from the MCU and the "funct" bits from the raw instruction. In essence, it's mostly just responsible for implementing the R-type instructions by decoding those "funct" bits (I-type instructions keep their operational assignments from the MCU via "regurgitation"). It does expand the number of instruction groupings, but these are all related to the newly interpreted R-type instructions. It's worth noting that there is no on/off control for either the ALU or MDU from either control unit (MCU or ACU), and thus, when either is not "officially" in use, it will just output garbage. This is fine for simplicity, but it may benefit power expenditure to gate access to the units based on expected deployment.

Arithmetic Logic Unit:

This module uses ACU control signals to perform basic arithmetic/logic operations using two input operands (reg+reg or reg+imm) and a shift amount input. It does not implement specific addition pathways, and instead leaves those decisions up to the hardware "compiler".

Multiplication & Division Unit:

This module uses ACU control signals to perform multiplication/division using two 32-bit input registers and splits the 64-bit output between two registers (Hi and Lo). At present, multiplication operations employ looping "barrel-addition" to execute in one cycle. The downside to this implementation is the effect on the overall CPU clock speed. It is not immediately obvious how much time is lost in comparison to a multi-cycled multiplication operation, since it saves cycles but hurts clock frequency. Division, at present, does not have the same explicit implementation pattern. For this reason, division does not have well-defined/known cycle behavior, and would likely be in need of an update for future uses (perhaps the same with multiplication). The MDU in totality also does not currently expose flags. Theoretically, it only requires a busy flag (ready = !busy) for stalls if multi-cycling is employed and perhaps a "DIV_ZERO" flag for errors, but I haven't attempted to implement either so far. With proper stalling behavior, this is generally not a problem, but the addition of such flags might reduce the number of stall cycles whenever MDU data is accessed if I did multi-cycle the operations.

Data Memory:

This module takes in the ALU output (interpreted as an address) and the 2nd data register (exposed by the Register File) as inputs and outputs a single 32-bit register. It also takes in clock, reset (stores are clocked operations), and the designated memory portion of the main control signal. It declares memory as a 4KB, word-addressable SRAM block and thus accesses portions using 10-bit addresses. For this reason, the addressed locations are interpreted by taking only the least 10 bits (number of words in 4KB) following a 2-bit offset from the address input. Load instructions read the 2-bit offset to determine which portion of the word in memory to extract for sub-word-sized outputs. Load-Byte instructions use both bits (4 bytes = 4 positions = 2 bits), whereas Load-Half instructions use only the greater offset bit (2 halfwords = 2 positions = 1 bit). This data partitioning is performed even when the instruction does not require it (for code simplicity). The reduced main control signal dictates what gets used (byte data, halfword data, or word data), and sign extensions for signed loads are carried out by repeating the top bit of the reduced data size enough times to fill the word. Stores use the address offset similarly, where the offset bits determine where to put the data to be stored ("centering" halfwords is not supported). This data marked for storage is read from the 2nd data register.

Hi/Lo Unit:

This clocked unit (takes clock and reset) takes the MDU outputs and portions of the main control signal as inputs and outputs the Hi and Lo registers (both 32 bits). When writes to hi/lo are enabled (as determined from the reduced control signal), the respective registers are updated with the clock (just like stores in data memory).

Writeback Multiplexer:

This special MUX uses the paths/destinations specified by the writeback portion of the main control signal to determine which input gets written to the Register File. It specifically picks between ALU outputs (general instructions), Data Memory outputs (loads), PC+4 (jump and link instructions), and Hi/Lo outputs (move from instructions).

Program Counter:

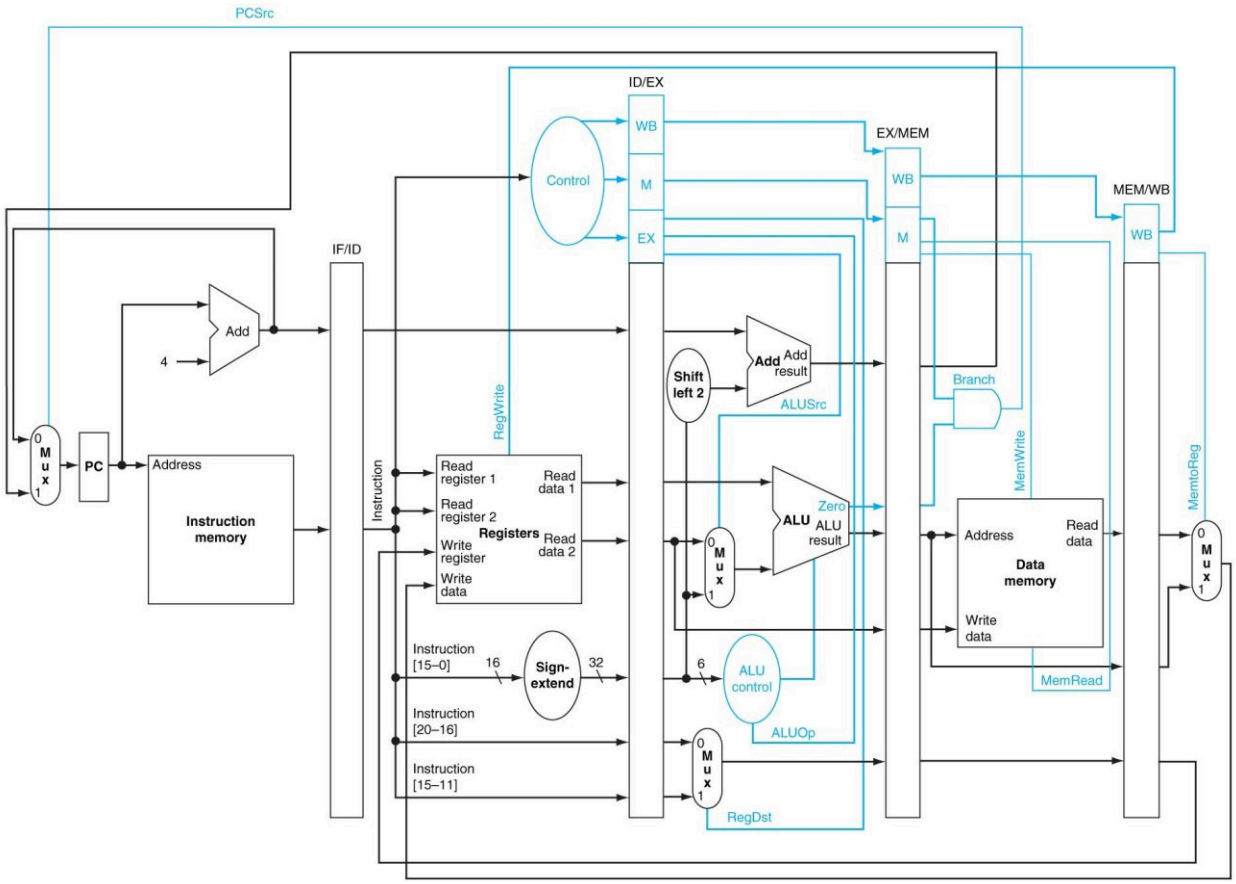
This clocked module updates the program counter to the Next PC input as long as updates are enabled. Next PC logic is handled separately in the CPU, and the enable input is tied to stalls (no stall = enable).

Instruction Memory:

This module takes the current PC as an input and outputs a 32-bit instruction. The instruction is pulled from the corresponding program memory file, with each instruction being assigned an address. Addresses are generated by declaring 4KB of word-addressed instruction memory and assigning a 2-bit offset (4 unit separation = “PC+4”). Upon generating addresses, instructions are loaded into each memory location using a special Verilog command (reads the program memory file referenced earlier), which removes the need to load instructions manually using a testbench.

Pipes:

Due to there being 5 pipeline stages, there are 4 corresponding “pipes” that transfer information between them. Each pipe is a clocked module that takes in the relevant “data to be transferred” from the first stage and outputs that same information when the transfer is enabled (via an enable input). To do this, the input and output variables are basically identical, save for stage identification “subscripts” (also, only one clock, reset, flush, and enable are needed for each pipe). Enable, as mentioned before, is handled exclusively by stall assessment (no stall = enable). Flush is similar, but slightly more complex, as flush is assigned when branching occurs (don’t process/flush the next instruction if a loop occurs). When flush is assigned, all variables are wiped (as implied by the name). Due to the implemented flush logic, this processor does not adhere to the typical MIPS branch delay slot procedure, as none can be present (flush takes precedent over enable). Beyond the absence of branch delay slots, the pipe architecture is effectively the same as the typical 5-stage MIPS design:



Variance between my derived architecture and the one pictured is discussed in the MCU. As a final note, there are some slight differences between specific pipes with and without forwarding, as forwarding induces the need to pass a few extra variables. This is to compare write addresses so that forwarding is actually executed when a write address in memory (or later) matches a read address in decode.

Hazard Unit:

This is the first module to have significant variance between implementations (obviously). Some minor "bloat" discrepancies occur as a result of writing one before the other. To begin:

1. No Forwarding:

    a. The write addresses, register write enable, and hi/lo write enable from the EX, MEM, and WB stages, in addition to the read addresses and various main control signals from ID, are taken as inputs, and the output is a single-bit stall signal. First, it uses those "oddly specific" control signals to determine if a specific register will actually be used/accessed during the instruction (e.g., non-register jump instructions use neither). Next, it determines if there are matches between the read addresses and any write addresses further down the pipeline. Then, if a register is used, and its address matches a write address later down the pipeline, and write is enabled for that instruction, a hazard is assessed based on the location/stage at which this conflict would occur (this process is carried out for both read addresses). In addition to the traditional stage processes, a Hi/Lo stall is also assessed (using similar logic). Finally, if any hazard is identified, the stall flag is set to high.

2. Forwarding:

    a. The write addresses and "memory read?" control signal from EX, along with the read addresses and "uses register?" flags from ID, are taken as inputs, while the output is given as a single-bit stall. Additionally, the hi/lo write enables from EX and MEM are designated as inputs. Two types of stalls are declared in Load-Use and Hi/Lo-Access. Load-Use stalls are assessed when the memory read flag is enabled for the instruction in EX, the corresponding uses register flag is enabled for the instruction in EX, and the read address from ID matches the write address in EX (this goes for both read addresses). Hi/Lo-Access stalls are assessed when hi/lo reads are enabled, and hi/lo writes are enabled for any stage later than decode (excluding writeback). If either stall is declared, then the global stall is set to high.

In both versions, the Hi/Lo-Access hazard/stall is relatively underdeveloped (as mentioned before), though the forwarding version removes the writeback check due to a perceived "unnecessary level of conservatism" that would result from its inclusion. Additionally, the transition from seemingly random control signals to a "uses register?" flag was made in the forwarding version since that same flag became useful for other CPU function calls and internal logic. As such, the old logic for register use found in the "original" hazard unit (without forwarding) lives in the CPU module. Notes aside, it is fairly evident that forwarding reduces stalls dramatically, which implies its greater efficiency.

Forwarding Unit:

The only unit exclusive to a version (for obvious reasons), this module takes the read addresses from EX (hence the change in the ID/EX pipe), the "uses register?" flags from EX, and the write addresses/enables from MEM and WB as inputs, and outputs the 2-bit signals "forward A/B". Internally, it declares 3 types of forwarding (none, from memory, from writeback), and, assuming registers are being used (the flag), it assigns a forwarding type based on where read/write address matches are identified. To prioritize newer updates, memory forwarding takes precedence over writeback forwarding. In the CPU module, this signal is then interpreted to facilitate proper data transfer (create the intended forward-chaining behavior).

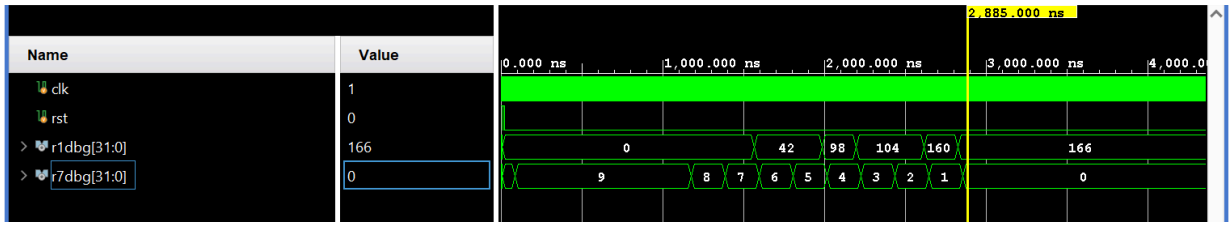
Central Processing Unit:

The central purpose of this (clocked) module is to interconnect/weave all the others together. Unfortunately, this makes it quite the eyesore. That said, if we ignore function calls and wire/register initializations, there are only a few extra bits of logic that require explanation. Among these is the calculation/determination of Next PC (without which nothing would run). First, the branch and jump addresses are configured (according to the reference material). Then, the "branch taken?" flag is set based on whether any branch conditions are satisfied. For BEQ and BNE, this compares the ALU output (in EX) to 0. For BLEZ and BGTZ, this compares the signed read data 1 register value to 0 (note that I am implying that forwarded data replaces the data in this register as applicable). When the branch taken flag is high, Next PC will take the value of the branch target (address). If instead the jump flag (via the main control signal) is high and the register jump flag bit is also high, Next PC will take the value stored in read data 1. Otherwise, if the jump flag alone is high (register jump flag is low), Next PC will take the value of the jump target (address). Finally, if none of these external conditions are met, Next PC will take PC+4 as its new value. While it may have been possible to position this logic elsewhere, it felt the most convenient to place it within this module. The next special logic executed in the CPU pertains to forwarding (as referenced above). In accordance with the forwarding types, we overwrite the value in the read data registers in EX with the value returned from the Writeback Multiplexer, either in the MEM stage or the WB stage (this portion obviously does not pertain to the version without forwarding). We initialize the Writeback Multiplexer twice (once in each mentioned stage), specifically for this purpose. Finally, we have to configure the flush and enable logic using the stalls from the Hazard Unit. For IF/ID, flush is set as high if branching or jumping occurs (irrespective of type), since we do not want to process instructions that are not from the branched/jumped location. IF/ID enable is set as the negation of stall (if no stalls, proceed). PC enable is the same, whereas ID/EX, EX/MEM, and MEM/WB enable are all fixed to high (there may be occasions to change this, but I currently do not have one). Logically, this enable discrepancy occurs since we typically only stall instructions from passing through to EX until the relevant data has been passed through from WB. Metaphorically, this is equivalent to flushing the toilet before you use it. Similarly, ID/EX flush is set to high when stall is high or when branching/jumping occurs (we don't want to process instructions with outdated values or process instructions from the wrong address). Finally, for similar reasons to enable, EX/MEM and MEM/WB flush are set to low (no reason to change

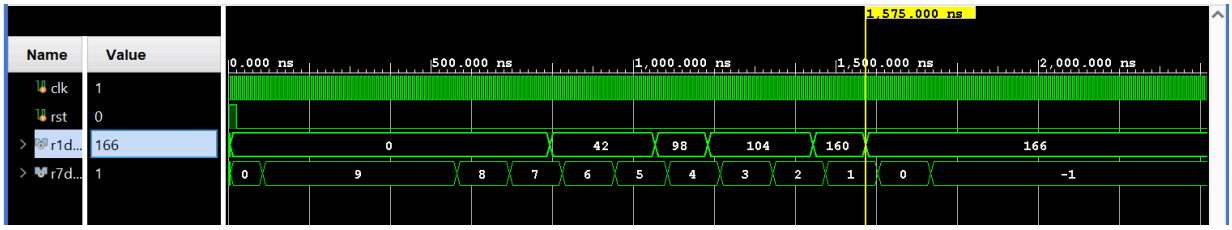
this presently). Forwarding introduces one change to this logic, since forwarding advances the processing of non-register jump instructions by a full cycle, and thus, must handle jump logic for ID as well as EX. In simple terms, if there are no present stalls and a non-register jump is ordered, IF/ID flush is also set to high. As a final note, I did mention that this module contains outputs for debugging, which is partially true. I temporarily instated outputs to read data contained in the Register File Unit, but these are both easily added and removed. Inclusion is not necessary in the slightest, and it is very easy to change which registers get read.

Results:

Without Forwarding:



With Forwarding:



Comparing the performances, we see a roughly 1.825 times speedup by including forwarding (2875 ns vs 1575 ns). Likewise, if each cycle is 10 ns, forwarding saves 130 cycles.

Final Notes:

Any other modules not elaborated upon are no more than basic MUXs, with the partial exception of the Register Destination MUX, whose purpose and function are outlined clearly in the MCU section (the function is identical to its prescribed description). Special thanks to Sam Altman for making a great tool for debugging (and I sup[1]pose self-teaching).

[1] **References:** MIPS Encoding Reference